

# 详细设计规范

## 费控云差旅平台

|                                                                                                                             |                   |
|-----------------------------------------------------------------------------------------------------------------------------|-------------------|
| 文件状态：<br><br>草稿 <input type="checkbox"/><br><br>修改 <input type="checkbox"/><br><br>正式发布 <input checked="" type="checkbox"/> | 所属项目编号：8          |
|                                                                                                                             | 版 本：1             |
|                                                                                                                             | 撰 写 人:徐文卓，徐鹤文，陈昱桦 |
|                                                                                                                             | 完成日期：2026-5-25    |
|                                                                                                                             | 发布日期：2026-5-26    |

目录

|                                |   |
|--------------------------------|---|
| 目录 .....                       | 1 |
| 1.非功能性要求 .....                 | 3 |
| 可靠性 .....                      | 3 |
| 高性能 .....                      | 3 |
| 可维护性 .....                     | 3 |
| 安全性 .....                      | 4 |
| 1. 术语定义 .....                  | 4 |
| 2. 功能性需求描述 .....               | 5 |
| 2.1 需求概述 .....                 | 5 |
| 2.2 业务全景图 .....                | 6 |
| 3 功能详细设计 .....                 | 6 |
| 3.1 用户登录（AuthController） ..... | 6 |
| 3.1.1 功能内容 .....               | 6 |
| 3.1.2 实现逻辑 .....               | 6 |
| 3.1.3 异常处置 .....               | 7 |
| 3.2 报销单列表查询 .....              | 7 |
| 3.2.1 功能内容 .....               | 7 |
| 3.2.2 实现逻辑 .....               | 7 |
| 3.2.3 异常处置 .....               | 8 |
| 3.3 报销单草稿创建与编辑 .....           | 8 |
| 3.3.1 功能内容 .....               | 8 |
| 3.3.1 实现逻辑 .....               | 9 |
| 3.3.3 异常处置 .....               | 9 |
| .....                          | 9 |

|                                                                                 |    |
|---------------------------------------------------------------------------------|----|
| 3.4 报销单状态流转 .....                                                               | 10 |
| 3.4.1 功能内容 .....                                                                | 10 |
| 3.4.2 实现逻辑 .....                                                                | 10 |
| 3.4.3 异常处置 .....                                                                | 10 |
| .....                                                                           | 11 |
| 3.5 主数据查询 .....                                                                 | 11 |
| 3.5.1 功能内容 .....                                                                | 11 |
| 3.5.2 实现逻辑 .....                                                                | 11 |
| 4 技术实现设计 .....                                                                  | 12 |
| 4.1 系统结构设计 .....                                                                | 12 |
| 4.1.1 后端模块划分 .....                                                              | 13 |
| 4.2 接口及核心类设计 .....                                                              | 14 |
| 4.2.1 核心 Service 类 .....                                                        | 14 |
| 核心实体关系 (ER) .....                                                               | 14 |
| 与前端的交互 .....                                                                    | 14 |
| 与第三方的交互 .....                                                                   | 15 |
| 5 关键技术点 .....                                                                   | 15 |
| 5.1 并发编程 .....                                                                  | 15 |
| 5.2 事务控制 .....                                                                  | 16 |
| 5.2.1 数据库局部事务 .....                                                             | 16 |
| 5.2.2 分布式事务 .....                                                               | 16 |
| 5.3Job 使用 .....                                                                 | 16 |
| 5.4 权限控制 .....                                                                  | 16 |
| 5.5 Redis 使用 .....                                                              | 17 |
| 5.6 敏感信息处理 .....                                                                | 17 |
| 5.7 错误码使用 .....                                                                 | 17 |
| 5.8 异动日志 .....                                                                  | 17 |
| 5.9 大数据量问题 .....                                                                | 18 |
| 5.10 缓存/MQ/重试机制 .....                                                           | 18 |
| 6.数据库设计数据库设计 .....                                                              | 18 |
| 6.1 数据库表设计 .....                                                                | 18 |
| 6.1.1 系统基础数据表 (sys_*) .....                                                     | 18 |
| 6.1.2 业务基础数据表 (biz_*) .....                                                     | 19 |
| 6.1.3 报销业务数据表 (fk_reim_*) .....                                                 | 19 |
| 6.2 数据库访问模块设计 .....                                                             | 19 |
| 7.WBS (工作分解结构) .....                                                            | 20 |
| 8.附录 1 接口定义明细 .....                                                             | 21 |
| 8.1 附录 1.1 认证接口 .....                                                           | 21 |
| 1.1. 附录 1.2 报销单接口 .....                                                         | 21 |
| 1.2. 附录 1.3 主数据接口 .....                                                         | 23 |
| 1.3. 附录 1.4 全局响应格式 .....                                                        | 23 |
| 1. 所有接口遵循此统一格式。HTTP 状态码与业务错误码独立：HTTP 200 可携带业务错误 (如 code=401 表示业务层认证失败) 。 ..... | 23 |

## 开发须知

详细设计是在充分理解需求的基础上，进行设计文档的编写，设计文档应充分说明复杂功能、核心功能关键实现方法、步骤、路径，应体现设计者对于需求的理解，以及知道如何实现，

设计文档也是指导实现者如何实现的参考指南。详细设计不仅要关注功能性需求的设计，同时也应该包含非功能性需求的设计，而我们往往容易忽略非功能性需求的设计，因此，这里特意予以强调。

## 1.非功能性要求

### 可靠性

可靠性指软件在异常情况下或在被非法、非常规使用时维持自身功能的能力。主要体现在容错和健壮性这两个方面。

容错指软件发生故障时仍保持正常运行的能力。它保证软件能在异常情况下正常运行，并在内部完成故障的修复工作。修复完成后，软件需要继续或从头开始执行异常位置的操作。

健壮性是保护软件不受非正常使用方式或非法输入影响的能力。具备该能力后，不论怎样的使用方式，软件都能准确迁移至系统定义的状态。

例如：

- 分布式系统在发生通信异常时会先暂时切断连接，等问题修复完成后再重新连接，恢复软件的运行；
- 系统缺陷率每1,000小时最多发生1次故障；
- 因软件系统的失效而造成无法完成业务的概率要小于5‰。

### 高性能

性能是系统或组件在给定的限制条件（如速度、精度或内存使用）内完成其指定功能的程度。性能表现是衡量软件质量的重要指标，在需求分析和系统设计阶段就必须充分考虑性能因素。性能指标主要包括响应时间、并发数、资源使用率等。简单地说，性能需求体现了系统如何“多快好省”地实现客户的功能需求。

例如：

- 响应时间：在95%的情况下，一般时段响应时间不超过1.5秒，高峰时段不超过4秒；在网络畅通时，电子地图刷新时间不超过10秒；
- 并发数：系统可以同时满足10,000个用户请求；
- 资源使用率：CPU占用率 $\leq 50\%$ ，内存占用率 $\leq 50\%$ 。

### 可维护性

功能在应对需求变更时，实现业务扩展时，能否比较方便地满足复杂多变的需

求，一般要在设计上，要进行灵活的设计，以实现业务上的持续迭代，保证一定的可维护性。

例如：

- 从接到修改请求后，对于普通修改应在1~2天内完成；
- 对于评估后为重大需求或设计修改应在1周内完成；
- 90%的BUG修改时间不超过1个工作日，其他不超过2个工作日。

## 安全性

安全性指产品消除潜在风险的能力和对风险的承受能力。包括保密性、可靠性和完整性三个子特性。保密性指数据不能被授权用户以外的任何人访问的能力。可靠性指授权用户可以不受阻止的访问数据、与其它软件的兼容的能力和产品的强壮度。完整性指按预期目标完成任务的能力。

一般分为程序安全、系统安全、数据安全。程序安全指开发的程序是否是安全的，程序上有没有安全的漏洞，例如Web开发中服务器代码没有对输入的参数进行验证，从而导致客户端机器人轻易的获取数据。系统安全指系统整体的安全，例如安全的粒度，未经授权的用户是否可以轻易的访问非法的数据等。数据安全是对数据的保护，数据库中数据有没有做审核，用户之间是否会共享数据等。

例如：

- 严格权限访问控制，用户在经过身份认证后，只能访问其权限范围内的数据，只能进行其权限范围内的操作；
- 提供运行日志管理及安全审计功能，可追踪系统的历史使用情况；
- 能经受来自互联网的一般性恶意攻击。如病毒（包括木马）攻击、口令猜测攻击、黑客入侵等；

## 1. 术语定义

| 缩写/术语        | 全称                                   | 说明                                 |
|--------------|--------------------------------------|------------------------------------|
| JWT          | JSON Web Token                       | 无状态身份认证令牌，基于 HMAC-SHA256 签名        |
| MyBatis-Plus | MyBatis 增强工具                         | 提供 Lambda 查询构造器、分页插件和乐观锁插件的 ORM 框架 |
| PBKDF2       | Password-Based Derivation Function 2 | Key<br>基于口令的密钥派生算法，用于密码哈希存储        |
| DTO/VO       | Data Transfer Object / View          | Java record 类，分别用于入参和出参的数据封装       |

|              | Object                   | 装                                     |
|--------------|--------------------------|---------------------------------------|
| WBS          | Work Breakdown Structure | 工作分解结构，用于项目任务划分和进度管理                  |
| Element Plus | Vue 3 UI 组件库             | 提供表格、表单、对话框等企业级 UI 组件                 |
| Pinia        | Vue 状态管理库                | Vue 3 官方推荐的状态管理方案                     |
| HikariCP     | 高性能 JDBC 连接池             | Spring Boot 默认连接池，提供极速连接获取            |
| SY           | 报销单号前缀                   | 报销单号格式为 "SY" + 10 位数字（如 SY0000000001） |

来自于产品或实施的需求描述，只谈需求，不谈任何设计实现。

## 2. 功能性需求描述

### 2.1 需求概述

费控云差旅平台是为企业提供差旅费用管控的 Web 应用系统，核心功能包括差旅报销单的全生命周期管理。系统包含以下四个核心需求：

#### 需求 1：用户登录与身份认证

系统用户通过用户名和密码进行登录认证，登录成功后获得 JWT 令牌，用于后续所有 API 请求的身份验证。令牌有效期 480 分钟，超时后需重新登录。

#### 需求 2：报销单全生命周期管理

支持报销单的创建（草稿）、编辑、提交审批、审批通过、撤回、作废、复制和删除等全流程操作。每个报销单包含：基本信息、出行行程、补助明细和费用分摊四个部分。

#### 需求 3：主数据管理

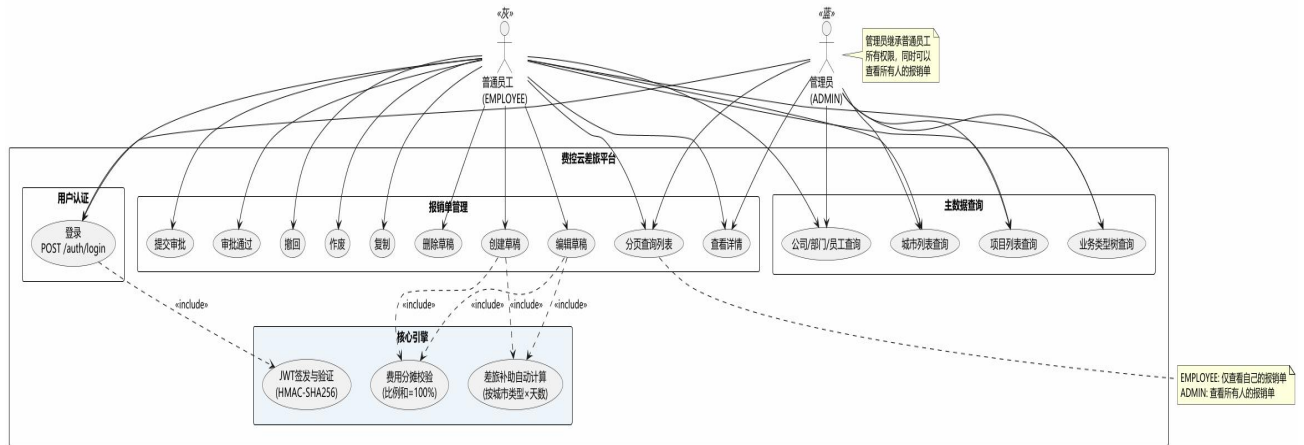
提供公司、部门、员工、业务类型（三级树形结构）、城市、项目等基础数据的查询接口，供前端下拉选择使用。业务类型支持级联选择（如：员工差旅活动 → 境内出差 → 项目出差）。

#### 需求 4：补助自动计算

根据行程天数和目的地城市类型自动计算餐费补助、交通补助和通讯补助的标准金额。城市类

型分为三类：一类城市 100 元/天，二类城市 80 元/天，三类城市 50 元/天。交通和通讯补助统一为 40 元/天。

## 2.2 业务全景图



## 3 功能详细设计

### 3.1 用户登录（AuthController）

#### 3.1.1 功能内容

- 前端登录页面收集用户名和密码，调用 POST /api/auth/login。
- 后端 AuthService 接收 LoginDTO，查询 sys\_user 表验证用户名和密码。
- 密码验证使用 PasswordHasher 进行 PBKDF2-SHA256 哈希比对。
- 验证通过后 JwtTokenService 生成 JWT 令牌，返回给前端。
- 前端将 Token 存入 localStorage，后续请求通过 Axios 拦截器自动附加 Authorization 头。

#### 3.1.2 实现逻辑

- (1) 前端提交 POST /api/auth/login，携带 username 和 password。
- (2) AuthController 委托 AuthService.login()处理。
- (3) AuthService 通过 UserMapper 查询 sys\_user 表，验证用户存在且 enabled=1。
- (4) PasswordHasher 解析存储的哈希格式 (pbkdf2\_sha256\$iterations\$salt\$hash)，使用 JCE 的 PBKDF2WithHmacSHA256 重新计算并比对。

(5) JwtTokenService 创建 JWT: Header(alg:HS256, typ:JWT) + Payload(sub, uid, eid, roles, iat, exp), 使用 HMAC-SHA256 签名。

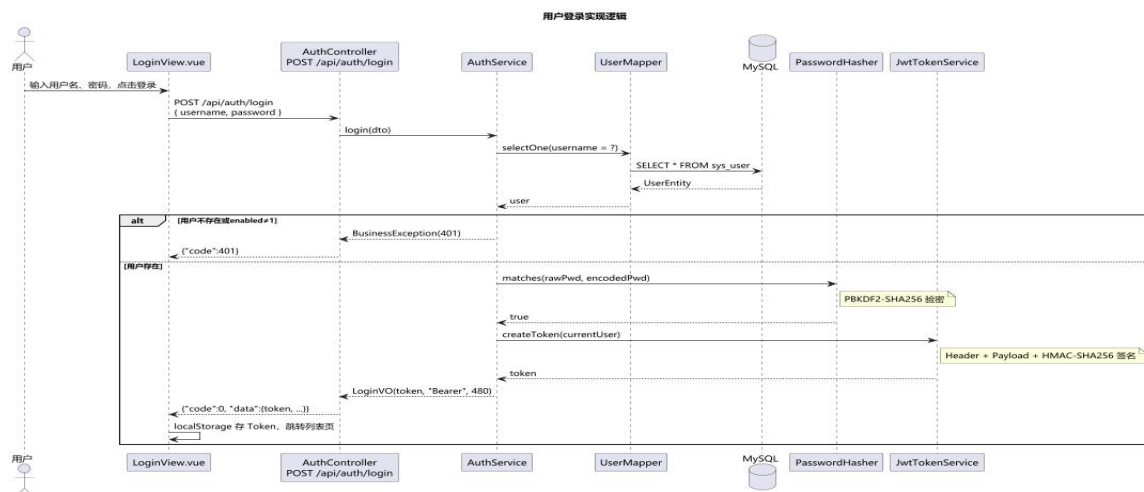
(6) 返回 LoginVO: token + tokenType("Bearer") + expiresInMinutes(480)。

(7) SecurityConfig 中/api/auth/login 路径 permitAll(), 其余请求需认证。

(8) JwtAuthenticationFilter 从请求头解析 Bearer Token , 还原 CurrentUser 并注入 SecurityContext。

### 3.1.3 异常处置

- 用户名不存在或密码不匹配: 抛出 BusinessException(401, "用户名或密码错误")。
- Token 过期: JwtTokenService 检测 exp 字段, 抛出 BusinessException(401), 前端清除 Token 并跳转登录页。
- Token 签名无效: 抛出 BusinessException(401, "令牌签名无效")。



## 3.2 报销单列表查询

### 3.2.1 功能内容

- 报销单列表页展示当前用户的报销单分页数据, 每页默认 10 条。
- 支持按单号、标题、事由、公司、部门、报销人、业务类型进行多条件组合筛选。

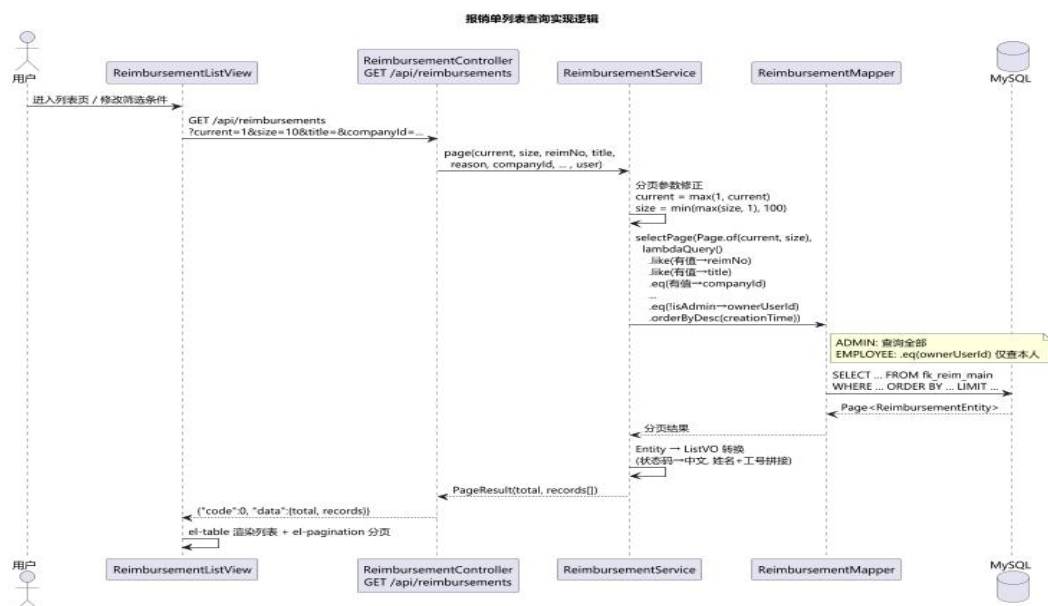
### 3.2.2 实现逻辑

(1) 前端 GET /api/reimbursements?current=1&size=10&...。

- (2) ReimbursementController.page()接收分页参数和筛选条件，通过@AuthenticationPrincipal 注入 CurrentUser。
- (3) ReimbursementService.page()使用 MyBatis-Plus Page + LambdaQueryWrapper 构建动态查询。
- (4) 非管理员用户自动过滤 owner\_user\_id=当前用户 ID，实现数据隔离。
- (5) 结果按 creation\_time 降序排列，返回 PageResult<ReimbursementListVO>。

### 3.2.3 异常处置

- 分页参数异常：Service 层对 page/size 做钳位（current 最小 1，size 最小 1 最大 100）。
- 数据库异常：GlobalExceptionHandler 捕获，返回 500+通用错误消息。



## 3.3 报销单草稿创建与编辑

### 3.3.1 功能内容

- 创建草稿：POST /api/reimbursements/drafts，首次创建自动生成单号。
- 编辑草稿：PUT /api/reimbursements/{id}/draft，仅草稿状态可编辑。
- 包含四个数据块：基本信息、行程列表、补助明细、费用分摊。
- 补助金额根据城市类型自动计算标准：一类 100 元、二类 80 元、三类 50 元，交通和通讯各 40 元。

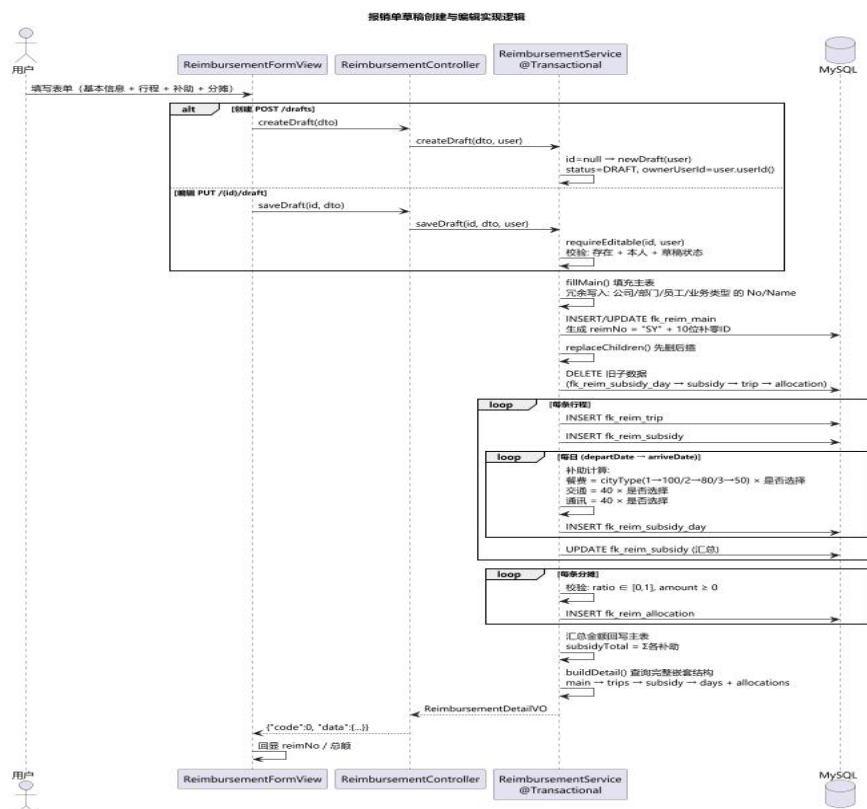


### 3.3.1 实现逻辑

- (1) Service 层使用@Transactional 保证原子性。
- (2) 编辑时采用"先删后插"模式（replaceChildren）：先级联删除旧子数据，再插入新子数据。
- (3) 行程验证：出行人/城市存在性、到达日期 $\geq$ 出发日期、到达日期 $\leq$ 当前日期、同一出行人行程日期不重叠。
- (4) 补助验证：每日补助日期在行程范围内、同行程日期不可重复、金额在 0 到标准之间。
- (5) 费用分摊验证：公司存在性、分摊比例 0~1、分摊金额非负。
- (6) 汇总计算：subsidy\_total/meal/transportation/phone 等汇总值回写主表。

### 3.3.3 异常处置

- 非草稿状态编辑：抛出 BusinessException("仅草稿报销单可编辑")。
- 行程日期重叠：同一出行人行程日期区间有交集时抛出 BusinessException。
- 分摊验证失败：分摊比例合计 $\neq$ 100%或金额合计 $\neq$ 补助总额时拒绝提交。



## 3.4 报销单状态流转

### 3.4.1 功能内容

状态枚举与流转规则：

| 状态   | 编码 | 允许的流转操作                        |
|------|----|--------------------------------|
| 草稿   | 0  | → 提交审批(→3); → 删除               |
| 审批通过 | 1  | → 作废(→2)                       |
| 已作废  | 2  | (终态, 不可流转)                     |
| 审批中  | 3  | → 审批通过(→1); → 撤回(→0); → 作废(→2) |

### 3.4.2 实现逻辑

所有状态变更方法均使用@Transactional。核心校验逻辑：

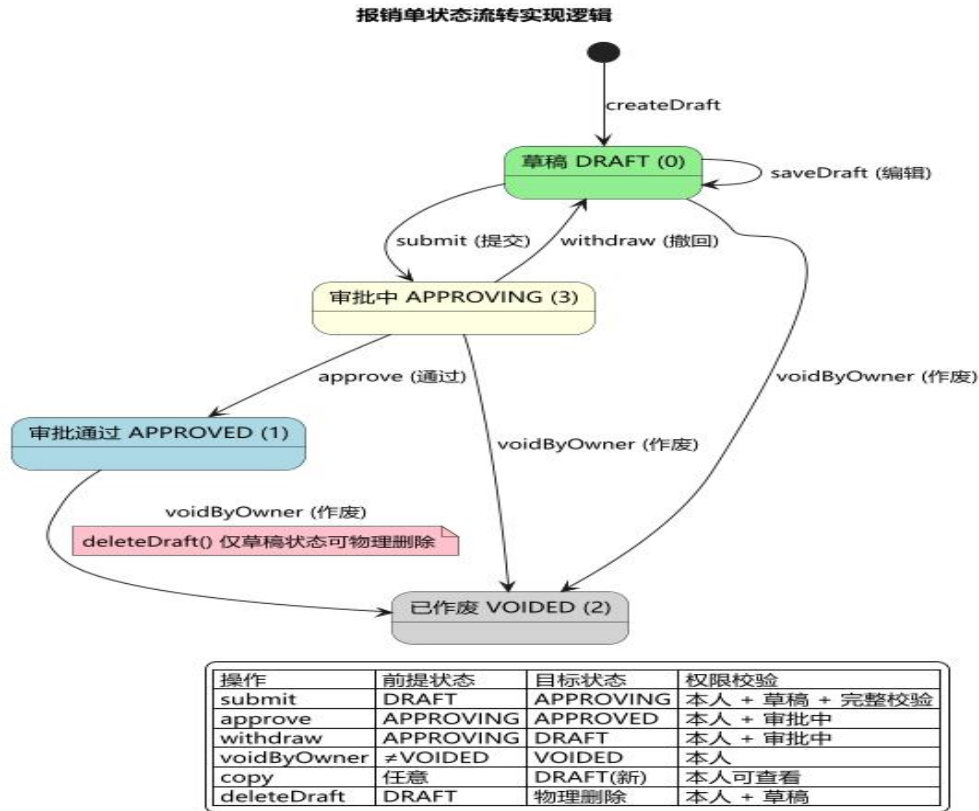
- submit(): 验证必填字段 + 行程/分摊完整性后, 状态 0→3。
- approve(): 验证状态=3, 状态 3→1。
- withdraw(): 验证状态=3, 状态 3→0。
- voidByOwner(): 验证状态≠2, 任意非作废→2。
- copy(): 读取详情, 复制基本信息+行程+补助+分摊生成新草稿。
- deleteDraft(): 验证状态=0 且为本人数据, 级联删除。

### 3.4.3 异常处置

- 状态校验：每个操作前严格校验当前状态, 不合法则抛出 BusinessException 说明原因。

- 权限校验：requireOwned() 验证 owner\_user\_id, 非本人数据抛出 BusinessException(403)。

- 数据库外键设置 ON DELETE CASCADE：删除主表自动清理 trip/subsidy/subsidy\_day/allocation。



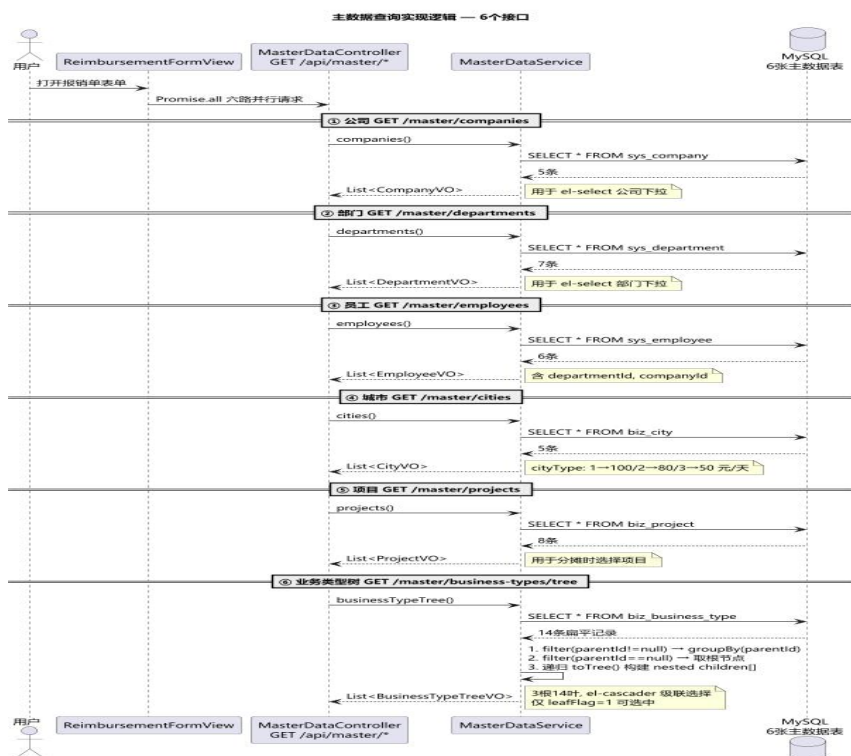
## 3.5 主数据查询

### 3.5.1 功能内容

- 提供 6 个主数据 GET 接口：公司、部门、员工、城市、项目、业务类型树。
- 全部返回全量数据（无分页），供前端下拉框和级联选择器使用。
- 业务类型以递归树形结构返回，parent\_id 构建父子关系，leaf\_flag 标识叶子节点。

### 3.5.2 实现逻辑

- (1) MasterDataController 路由前缀/api/master，委托 MasterDataService 处理。
- (2) MasterDataService 使用 Wrappers.lambdaQuery()全表查询。
- (3) 业务类型树构建：查询全部→按 parent\_id 分组→从 parent\_id=null 的根节点递归构建树。
- (4) 所有 VO 使用 Java record 类，不可变且自动生成访问器。



## 4 技术实现设计

### 4.1 系统结构设计

后端技术栈:

运行环境: Java 17 + Spring Boot 4.0.6 + Tomcat 11 (嵌入式)

安全框架: Spring Security 7 + 自实现 JWT (HMAC-SHA256 签名)

ORM 框架: MyBatis-Plus 3.5.15 + MySQL 8.0 + HikariCP 7.0.2

校验: Jakarta Validation (Hibernate Validator 9)

JSON 序列化: Jackson 3.1.2

前端技术栈:

运行环境: Node.js 22 + Vite 8

UI 框架: Vue 3.5 + TypeScript 6.0 + Element Plus 2.11

状态管理: Pinia 3.0 + 路由: Vue Router 5.0

HTTP 客户端: Axios 1.13

## 4.1.1 后端模块划分

| 一级模块          | 二级模块      | 三级模块                    | 职责说明                                          |
|---------------|-----------|-------------------------|-----------------------------------------------|
| config        | security  | SecurityConfig          | Spring Security 安全配置，定义过滤链                    |
| config        | security  | JwtTokenService         | JWT 令牌创建 (HMAC-SHA256)与解析验证                   |
| config        | security  | JwtAuthenticationFilter | OncePerRequestFilter，解析 Bearer Token          |
| config        | security  | CurrentUser             | Java record，用户身份信息载体                          |
| config        | exception | BusinessException       | 业务异常类，携带错误码和消息                                |
| config        | exception | GlobalExceptionHandler  | @RestControllerAdvice 全局异常处理                  |
| config        | mybatis   | MybatisPlusConfig       | 分页插件与乐观锁插件配置                                  |
| controller    | -         | AuthController          | 认证接口：POST /api/auth/login                     |
| controller    | -         | ReimbursementController | 报销单 CRUD 全生命周期接口 (11 个)                       |
| controller    | -         | MasterDataController    | 主数据查询接口(6 个 GET 接口)                           |
| service       | -         | AuthService             | 登录验证逻辑 + JWT 签发                               |
| service       | -         | ReimbursementService    | 报销单核心业务(803 行)，含状态流转/补助计算                     |
| service       | -         | MasterDataService       | 基础数据查询 + 业务类型树构建                              |
| service       | -         | PasswordHasher          | PBKDF2-SHA256 密码哈希验证组件                        |
| mapper        | -         | 12 个 Mapper 接口          | 继承 BaseMapper<T>，无需 XML 配置                    |
| entity/dto/vo | -         | 数据对象包                   | 12 个 Entity + 5 个 DTO record + 16 个 VO record |

## 4.2 接口及核心类设计

### 4.2.1 核心 Service 类

#### ReimbursementService

报销单核心服务（803 行），管理报销单全生命周期。核心方法：`page()`分页查询、`detail()`详情、`createDraft()/saveDraft()`草稿 CRUD、`submit()/approve()/withdraw()/voidByOwner()`状态流转、`copy()`复制、`deleteDraft()`删除。内部类 `Summary` 使用累加器模式汇总补助金额。

#### JwtTokenService

自实现 JWT 服务，不依赖第三方 JWT 库。`createToken()`构建三段式 JWT 并用 HMAC-SHA256 签名；`parseToken()`验证签名+过期时间后还原 `CurrentUser`。

#### MasterDataService

主数据查询服务，提供 6 种基础数据的全量查询。`businessTypeTree()`使用递归算法从扁平数据构建三级树形结构。

## 核心实体关系（ER）

报销主表（`fk_reim_main`）为聚合根，关联关系如下：

- `fk_reim_main` → `fk_reim_trip`（1:N）：一个报销单可包含多条行程
- `fk_reim_trip` → `fk_reim_subsidy`（1:1）：每条行程对应一条补助汇总
- `fk_reim_subsidy` → `fk_reim_subsidy_day`（1:N）：每条补助包含多天每日明细
- `fk_reim_main` → `fk_reim_allocation`（1:N）：一个报销单可有多条费用分摊
- `fk_reim_main` → `sys_user`（N:1）：关联数据所有者（`owner_user_id`）
- 所有子表外键设置 ON DELETE CASCADE，删除主表自动级联清理。

## 与前端的交互

前后端通过 RESTful API 交互，路径前缀 `/api`，JSON 格式。前端通过 Vite 代理（`/api` → `localhost:8088`）转发。

| 功能   | 方法   | 接口路径                             | 前端调用位置                           |
|------|------|----------------------------------|----------------------------------|
| 用户登录 | POST | <code>/api/auth/login</code>     | <code>authStore.login()</code>   |
| 报销列表 | GET  | <code>/api/reimbursements</code> | <code>getReimbursements()</code> |

|      |        |                                   |                         |
|------|--------|-----------------------------------|-------------------------|
| 报销详情 | GET    | /api/reimbursements/{id}          | getReimbursement()      |
| 创建草稿 | POST   | /api/reimbursements/drafts        | createDraft()           |
| 保存草稿 | PUT    | /api/reimbursements/{id}/draft    | saveDraft()             |
| 提交审批 | POST   | /api/reimbursements/{id}/submit   | submitReimbursement()   |
| 审批通过 | POST   | /api/reimbursements/{id}/approve  | approveReimbursement()  |
| 撤回   | POST   | /api/reimbursements/{id}/withdraw | withdrawReimbursement() |
| 作废   | POST   | /api/reimbursements/{id}/void     | voidReimbursement()     |
| 复制   | POST   | /api/reimbursements/{id}/copy     | copyReimbursement()     |
| 删除草稿 | DELETE | /api/reimbursements/{id}          | deleteReimbursement()   |
| 公司列表 | GET    | /api/master/companies             | getCompanies()          |
| 部门列表 | GET    | /api/master/departments           | getDepartments()        |
| 员工列表 | GET    | /api/master/employees             | getEmployees()          |
| 城市列表 | GET    | /api/master/cities                | getCities()             |
| 项目列表 | GET    | /api/master/projects              | getProjects()           |
| 业务类型 | GET    | /api/master/business-types/tree   | getBusinessTypes()      |

## 与第三方的交互

当前 V1.0 版本为独立部署的单体应用，不涉及第三方系统交互。所有功能在系统内部闭环完成。  
未来扩展点：企业微信审批流对接、财务系统数据同步（预留 Feign/HTTP 客户端扩展接口）。

## 5 关键技术点

### 5.1 并发编程

当前 V1.0 版本已在报销单主表 `fk_reim_main` 中加入 `version` 乐观锁字段，并通过 MyBatis-Plus `@Version` 与 `OptimisticLockerInnerInterceptor` 防止并发覆盖。前端打开报销单详情时保存当前 `version`，保存草稿时随 `ReimbursementDraftSaveDTO` 一并提交；后端保存前比对数据库最新 `version`，更新影响行数为 0 或版本不一致时返回 `BusinessException(409,`

"报销单已被他人修改，请刷新后重试")。报销单号仍使用数据库自增 ID 生成，避免并发冲突。

## 5.2 事务控制

### 5.2.1 数据库局部事务

核心写操作均使用 Spring 声明式事务（@Transactional）：

- createDraft()/saveDraft(): 保证主表+子表（行程/补助/分摊）的原子写入。
- replaceChildren(): 先删后插模式，在事务内完成子表数据全量替换。
- 所有状态变更方法（submit/withdraw/approve/void）均使用@Transactional。
- 事务传播行为使用默认 REQUIRED，确保嵌套调用在同一事务中。

### 5.2.2 分布式事务

当前 V1.0 为单体应用，不涉及分布式事务。后续如拆分为微服务，需引入 Seata 或基于 MQ 的最终一致性方案。

## 5.3 Job 使用

当前 V1.0 版本无定时任务。后续可能涉及：报销数据定期归档 Job（清理超期的已审批/已作废数据）；JWT 为无状态设计，无需服务端清理 Job。

## 5.4 权限控制

系统实现基于角色的访问控制（RBAC）：

- CurrentUser record 携带 userId/employeeId/username/roles ， 通过 @AuthenticationPrincipal 注入。
- 数据级权限：isAdmin()判断，非管理员自动过滤 owner\_user\_id。Service 层 requireOwned()校验归属。
- 操作级权限：每个状态流转方法校验当前状态是否允许目标操作。
- 前端路由守卫：beforeEach 检查 Token，未登录自动跳转/login。



## 5.5 Redis 使用

当前版本未使用 Redis（JWT 为无状态设计，无需 Session 存储）。后续引入建议：

- Key 前缀规范：travel:{模块}:{标识}（如 travel:cache:company:list）。
- 缓存主数据（公司/部门/城市等低变更数据），TTL 30 分钟。

## 5.6 敏感信息处理

- 用户密码：PBKDF2-SHA256 + 随机盐哈希存储，不可逆。
- JWT Secret：通过配置属性注入，生产环境以环境变量 TRAVEL\_JWT\_SECRET 覆盖。
- 数据库密码：通过环境变量 TRAVEL\_DB\_PASSWORD 覆盖，配置文件中不写明文。
- 前端 Token：存储于 localStorage，建议生产环境升级为 HttpOnly Cookie。

## 5.7 错误码使用

| 错误码 | HTTP 状态 | 说明        | 典型场景                 |
|-----|---------|-----------|----------------------|
| 0   | 200     | 成功        | 正常返回业务数据             |
| 400 | 400     | 参数/业务校验失败 | @Valid 校验不通过、业务规则不满足 |
| 401 | 401     | 认证失败      | 用户名密码错误、Token 无效/过期  |
| 403 | 403     | 无权限       | 操作他人数据或越权操作          |
| 404 | 404     | 资源不存在     | 报销单不存在               |
| 500 | 500     | 系统异常      | 未预期的运行时异常            |

• BusinessException 携带错误码和中文描述，GlobalExceptionHandler 统一转换为 Result<T>响应。

## 5.8 异动日志

系统通过以下字段记录关键操作：

- created\_at：数据创建时间（数据库默认 CURRENT\_TIMESTAMP）。
- update\_time：数据最后更新时间（Service 层显式设置 LocalDateTime.now()）。
- owner\_user\_id：数据所有者，创建时写入且不可修改。
- 关键状态变更（提交/审批/撤回/作废）同时更新 update\_time，可追踪完整操作时间线。

## 5.9 大数据量问题

V1.0 数据量预测：fk\_reim\_main 年增量约 10 万条，5 年累计 50 万条。当前已采取的措施：

- 分页查询使用 MyBatis-Plus Page 插件 + 索引(status+creation\_time 复合索引)。
- 子表查询按 main\_id 精确匹配 + 主键索引，避免全表扫描。
- 后续数据量增长方案：按月分区、1 年以上已审批数据归档至历史表。

## 5.10 缓存/MQ/重试机制

当前 V1.0 版本均未使用缓存、MQ 和重试机制。这些技术点将在业务规模扩大后按需引入：缓存层使用 Spring Cache + Redis 优化主数据读取；MQ 使用 RocketMQ/RabbitMQ 处理审批通知等异步任务（需保证消息幂等性）；重试机制使用 Spring Retry 处理外部接口调用失败场景。

# 6 数据库设计

## 6.1 数据库表设计

数据库名：travel，字符集：utf8mb4，排序规则：utf8mb4\_0900\_ai\_ci。共 12 张表，分为三类：

### 6.1.1 系统基础数据表（sys\_\*）

| 表名             | 说明    | 主键/唯一键                   | 关联关系                            |
|----------------|-------|--------------------------|---------------------------------|
| sys_company    | 公司信息表 | id PK / company_no UK    | -                               |
| sys_department | 部门信息表 | id PK / department_no UK | -                               |
| sys_employee   | 员工信息表 | id PK / employee_no UK   | FK→sys_company, sys_department  |
| sys_user       | 系统用户表 | id PK / username UK      | FK→sys_employee ; roles 存储角色字符串 |

## 6.1.2 业务基础数据表 (biz\_\*)

| 表名                | 说明       | 关键字段                             | 特殊设计                          |
|-------------------|----------|----------------------------------|-------------------------------|
| biz_business_type | 业务类型（树形） | business_type_no/name, parent_id | parent_id 自关联, leaf_flag 标识叶子 |
| biz_city          | 城市信息表    | city_no/name, city_type          | city_type: 1 一类/2 二类/3 三类城市   |
| biz_project       | 项目信息表    | project_no/name                  | 含"非项目类费用归集"兜底记录               |

## 6.1.3 报销业务数据表 (fk\_reim\_\*)

| 表名                  | 说明   | 核心字段                                  | 外键/级联                  |
|---------------------|------|---------------------------------------|------------------------|
| fk_reim_main        | 报销主表 | reim_no, version, 状态/金额/人员/部门/公司/业务类型 | FK→sys_user(owner)     |
| fk_reim_trip        | 行程明细 | 出行人/出发城市/到达城市/日期/说明                   | FK→main(CASCADE)       |
| fk_reim_subsidy     | 补助汇总 | 天数/申请金额/实补/餐费/交通/通讯                   | FK→main, trip(CASCADE) |
| fk_reim_subsidy_day | 每日补助 | 日期/城市/标准/选中标记/实际金额                    | FK→subsidy(CASCADE)    |
| fk_reim_allocation  | 费用分摊 | 公司/项目/比例 (DECIMAL 8,6)/金额             | FK→main(CASCADE)       |

## 6.2 数据库访问模块设计

- Entity 类使用 Lombok @Data + @TableName 注解，字段自动映射（下划线转驼峰）。
- Mapper 接口继承 BaseMapper<T>，自动获得 CRUD 方法，无需编写 XML 映射文件。
- Service 层使用 Wrappers.lambdaQuery() 构建类型安全的动态查询条件。
- 分页使用 Page<T> + selectPage() 方法。
- 全局配置：map-underscore-to-camel-case=true + id-type=auto（数据库自增主键）。

## 7.费控云差旅平台开发项目 WBS

| 序号 | 计划阶段 | 模块   | 任务名称          | 任务说明                                | 负责人 | 协助人 | 开始时间       | 结束时间       | 工期(天) | 当前状态 | 质量要求 | 备注   |
|----|------|------|---------------|-------------------------------------|-----|-----|------------|------------|-------|------|------|------|
| 1  | 后端开发 | 数据库  | 数据库设计与建库脚本    | 设计 12 张表, 编写建库脚本及示例数据               | 陈昱桦 |     | 2026-05-23 | 2026-05-23 | 0.5   | 已完成  |      |      |
| 2  | 后端开发 | 安全认证 | JWT 安全框架与登录接口 | Spring Security + JWT + PBKDF2 密码验证 | 陈昱桦 |     | 2026-05-23 | 2026-05-24 | 1     | 已完成  |      |      |
| 3  | 后端开发 | 报销单  | 报销单数据结构设计     | Entity、DTO、VO 定义及嵌套结构               | 陈昱桦 |     | 2026-05-24 | 2026-05-24 | 0.5   | 已完成  |      |      |
| 4  | 后端开发 | 报销单  | 报销单草稿创建与编辑    | 草稿的创建、保存, 含主表+行程+补助+分摊的原子写入         | 陈昱桦 |     | 2026-05-24 | 2026-05-25 | 1     | 已完成  |      |      |
| 5  | 后端开发 | 报销单  | 补助计算与费用分摊     | 按城市类型逐日计算补助, 分摊比例校验                 | 陈昱桦 |     | 2026-05-26 | 2026-05-27 | 1     | 进行中  |      |      |
| 6  | 后端开发 | 报销单  | 报销单状态流转       | 提交、审批、撤回、作废、复制、删除 6 种状态变更           | 陈昱桦 |     | 2026-05-26 | 2026-05-28 | 2     | 未完成  |      |      |
| 7  | 后端开发 | 项目基础 | 项目初始化与环境配置    | 前后端项目骨架搭建, 数据库初始化                   | 徐文卓 |     | 2026-05-23 | 2026-05-23 | 0.5   | 已完成  |      |      |
| 8  | 后端开发 | 主数据  | 主数据查询接口       | 公司/部门/员工/城市/项目/业务类型 6 个查询接口         | 徐文卓 |     | 2026-05-23 | 2026-05-24 | 1     | 已完成  |      |      |
| 9  | 后端开发 | 公共组件 | 统一异常处理        | 全局异常拦截、业务异常定义、参数校验                  | 徐文卓 |     | 2026-05-24 | 2026-05-24 | 0.5   | 已完成  |      |      |
| 10 | 测试联调 | 联调   | 前后端联调测试       | 接口联调、流程测试、Bug 修复                    | 徐文卓 |     | 2026-05-25 | 2026-05-26 | 0.5   | 已完成  |      | 全员配合 |
| 11 | 文档   | 文档   | 接口与设计文        | API 文档、数据库                          | 徐文  |     | 2026-05-27 | 2026-05-30 | 2     | 未开   |      |      |

| 序号 | 计划阶段 | 模块   | 任务名称    | 任务说明                        | 负责人 | 协助人 | 开始时间       | 结束时间       | 工期(天) | 当前状态 | 质量要求 | 备注 |
|----|------|------|---------|-----------------------------|-----|-----|------------|------------|-------|------|------|----|
|    |      |      | 档       | 文档、部署说明                     | 卓   |     |            |            |       | 始    |      |    |
| 12 | 前端开发 | 项目基础 | 前端项目初始化 | Vue3 项目骨架、路由、Axios 封装、Store | 徐鹤文 |     | 2026-05-23 | 2026-05-23 | 0.5   | 已完成  |      |    |
| 13 | 前端开发 | 登录   | 登录页与主布局 | 登录表单、Token 管理、AppShell 框架布局 | 徐鹤文 |     | 2026-05-23 | 2026-05-24 | 0.5   | 已完成  |      |    |
| 14 | 前端开发 | 报销单  | 报销单列表页  | 表格展示、分页、多条件筛选               | 徐鹤文 |     | 2026-05-24 | 2026-05-25 | 1     | 已完成  |      |    |
| 15 | 前端开发 | 报销单  | 报销单表单页  | 创建/编辑表单, 含行程、补助、分摊子组件       | 徐鹤文 |     | 2026-05-25 | 2026-05-26 | 1     | 进行中  |      |    |

## 8.附录 1 接口定义明细

### 附录 1.1 认证接口

| 项目   | 说明                                                                                  |
|------|-------------------------------------------------------------------------------------|
| 接口名称 | 用户登录                                                                                |
| 请求方式 | POST /api/auth/login                                                                |
| 认证要求 | 无需认证（SecurityConfig 中 permitAll）                                                    |
| 请求参数 | JSON: {"username":"demo", "password":"Travel@123"}                                  |
| 成功响应 | {"code":0, "data":{"token":"eyJ...", "tokenType":"Bearer", "expiresInMinutes":480}} |
| 失败响应 | {"code":401, "message":"用户名或密码错误"}                                                  |

### 附录 1.2 报销单接口

#### (1) 报销单列表查询：

| 项目 | 说明 |
|----|----|
|----|----|

|      |                                                                                                            |
|------|------------------------------------------------------------------------------------------------------------|
| 请求方式 | GET /api/reimbursements                                                                                    |
| 请求参数 | current(页码), size(每页条数), reimNo, title, reason, companyId, departmentId, reimbuserId, businessTypeId (均可选) |
| 成功响应 | {"code":0, "data":{"total":14, "current":1, "size":10, "records":[...]}}                                   |

## (2) 报销单详情:

| 项目   | 说明                                              |
|------|-------------------------------------------------|
| 请求方式 | GET /api/reimbursements/{id}                    |
| 成功响应 | {"code":0, "data":{"...完整报销单 (含行程/补助/分摊嵌套结构) }} |

## (3) 创建/编辑草稿:

| 项目   | 说明                                                                                                                                                                                                                                                                                                                                                                                 |
|------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 创建草稿 | POST /api/reimbursements/drafts                                                                                                                                                                                                                                                                                                                                                    |
| 编辑草稿 | PUT /api/reimbursements/{id}/draft                                                                                                                                                                                                                                                                                                                                                 |
| 请求体  | ReimbursementDraftSaveDTO:{reimbursementTitle, reimbuserId, reimDepartmentId, reimCompanyId, businessTypeId, businessTripReason, remarks, version, trips:[{travelerId, departCityId, arriveCityId, departDate, arriveDate, tripDescription, subsidyDays:[{subsidyDate, mealSelected, mealAmount, ...}]}], allocations:[{companyId, projectId, allocationRatio, allocationAmount}]} |

## (4) 状态流转接口:

| 操作   | 请求                                     | 前置状态→目标状态        |
|------|----------------------------------------|------------------|
| 提交审批 | POST /api/reimbursements/{id}/submit   | 草稿(0) → 审批中(3)   |
| 审批通过 | POST /api/reimbursements/{id}/approve  | 审批中(3) → 审批通过(1) |
| 撤回   | POST /api/reimbursements/{id}/withdraw | 审批中(3) → 草稿(0)   |
| 作废   | POST /api/reimbursements/{id}/void     | 非作废(≠2) → 已作废(2) |
| 复制   | POST /api/reimbursements/{id}/copy     | 任意状态 → 新草稿(0)    |
| 删除   | DELETE /api/reimbursements/{id}        | 草稿(0) → 删除       |

## 附录 1.3 主数据接口

| 接口    | 路径                                  | 返回示例                                                                     |
|-------|-------------------------------------|--------------------------------------------------------------------------|
| 公司列表  | GET /api/master/companies           | [[id, companyNo, companyName]]                                           |
| 部门列表  | GET /api/master/departments         | [[id, departmentNo, departmentName]]                                     |
| 员工列表  | GET /api/master/employees           | [[id, employeeNo, employeeName, departmentId, companyId]]                |
| 城市列表  | GET /api/master/cities              | [[id, cityNo, cityName, cityType]]                                       |
| 项目列表  | GET /api/master/projects            | [[id, projectNo, projectName]]                                           |
| 业务类型树 | GET /api/master/business-types/tree | [[id, businessTypeNo, businessTypeName, parentId, leaf, children:[...]]] |

## 附录 1.4 全局响应格式

| 字段      | 类型     | 说明                 |
|---------|--------|--------------------|
| code    | int    | 业务状态码：0=成功，其他见错误码表 |
| message | string | 提示信息，成功时为"success" |
| data    | T（泛型）  | 响应数据体，失败时为 null    |

**1.** 所有接口遵循此统一格式。**HTTP 状态码**与**业务错误码**独立：**HTTP 200**可携带业务错误（如**code=401**表示业务层认证失败）。